

Reinforcement learning of a steering model for a foiling sailboat D. Dilruwan¹ & W.

Peifeng¹

Benjamin Negrevergne² & Camille Negrevergne²

¹*ENSAI* ²*Nautia*

{peifeng.wang, dilruwan.dehiwalage-don}@eleve.ensai.fr
{camille, benjamin}@nautia.fr

February 8, 2026

Abstract

In the context of elite offshore racing, such as the Vendée Globe¹, traditional autopilot systems are limited by their inability to optimize trajectories amidst stochastic environmental changes. With the advent of hydrofoils, which reduce drag by lifting the hull, steering complexity has increased, necessitating controllers that can proactively manage stability and speed. This study presents a Deep Reinforcement Learning (DRL) framework developed for Nautia to determine optimal steering policies in a custom Gymnasium-based simulator. The agent’s task is defined as a Markov Decision Process (MDP) focused on upwind navigation under variable wind directions (90° to 140°) and speeds (10 to 18 knots). We evaluate and compare three distinct control strategies: a baseline PID controller, a native Multi-Layer Perceptron (MLP), and a novel 1D Convolutional Neural Network (1D CNN) architecture. Our empirical results demonstrate that the 1D CNN offers superior high-performance capabilities, achieving a peak “Course Made Good” (CMG) of **24.59 knots** in stress-test conditions (95° heading, 20 knots wind). Quantitatively, the 1D CNN outperformed the PID baseline in the majority of training-distribution scenarios, showing improvements of up to **+5.27%**. Conversely, the MLP baseline struggled with stability in high-wind regimes, often underperforming the PID by significant margins. While the 1D CNN showed vulnerability to “Out-of-Bounds” failures in light-wind boundary conditions (10 knots), it proved to be the most robust architecture for high-speed foiling, effectively extracting temporal features from wind fluctuations to provide a competitive steering policy for modern racing sailboats.

Keywords: Deep Reinforcement Learning, Autonomous Sailing, Foiling Sailboat, 1D Convolutional Neural Network, Steering Control, Markov Decision Process, Velocity Made Good, PID Controller

¹The Vendée Globe is a single handed non stop, unassisted round the world yacht race

Contents

1	Introduction	1
1.1	Autonomous Navigation and Control via Reinforcement Learning	1
1.2	Principles of Sailboat Dynamics	2
1.3	Problem Definition and Navigation Challenge	3
1.3.1	Simulation Environment: <i>BoatSimulator</i>	3
1.3.2	Baseline PID Performance and Navigation Limitations	3
2	State of the Art in Autonomous Sailing Agents	5
2.1	Implementation Framework and Training Protocol	5
2.2	Agent Architecture and Policy Network	5
2.3	Temporal Feature Extraction with 1D-CNNs	6
2.4	Training Protocol	7
2.4.1	Reward Design: CMG Optimization Objective	7
3	Results and Discussion	9
3.1	Training Convergence and Optimal Checkpoint Selection	9
3.2	Evaluation Framework and Performance Metrics	9
3.3	Steady-State Performance Comparison	10
3.4	Control Dynamics and Temporal Stability	11
3.5	Trajectory Analysis	12
4	Conclusion and Future Work	13
A	Complete Experimental Results Table	14
B	CMG Improvement Over Baseline	16
C	Complete Trajectory Comparison Across Architectures and Wind Conditions	16

1 Introduction

1.1 Autonomous Navigation and Control via Reinforcement Learning

The application of Deep Reinforcement Learning (DRL) in autonomous navigation and control is becoming increasingly ubiquitous. In the context of autonomous sailing, DRL addresses a high dimensional optimal control problem characterized by nonlinear dynamics and significant environmental uncertainties. The ability of a DRL agent to learn this control problem directly correlates with performance improvements in elite competitive environments, such as the solo round the world Vendée Globe². Racing environments often employ a specialized class of vessel known as a *sailboat*. Compared to traditional motorized vessels sailboats operate without an engine and they rely entirely on wind as their primary power source. Nevertheless, modern racing sailboats are equipped with sophisticated autopilot systems that are essential for both sailor safety and competitive performance. Figure 1 illustrates the IMOCA 60 class sailboat, representing the pinnacle of technological innovation and navigational complexity in offshore racing.



Figure 1: The IMOCA 60 *Malizia Seaexplorer*, one of the world’s fastest monohulls.

These sailboats transform navigation into a complex strategic problem in which every decision must reconcile nonlinear aerodynamic and hydrodynamic interactions with continuously evolving environmental conditions. One can naturally formulate this navigation task as a *Markov Decision Process (MDP)*. A key limitation of current systems is that they typically rely on closed loop control³ to maintain a fixed heading or apparent wind angle, without explicitly optimizing trajectory efficiency or speed. Under volatile conditions, human sailors often need to intervene manually to remain competitive. Maximizing the boat’s velocity toward an upwind target (the Course Made Good (CMG)⁴) requires continuous adaptation to stochastic environmental variables such as fluctuating wind vectors and changing sea states.

Reinforcement learning (RL) offers a natural framework for addressing the challenges of sequential decision making under uncertainty. In the traditional machine learning (ML) paradigm, supervised learning relies on labeled datasets to map inputs to known outputs, while unsupervised learning seeks to identify hidden patterns within unlabeled data. Deep Reinforcement Learning (DRL) in contrast, enables an agent to discover optimal behavior through trial and error. It operates as a sequential decision making framework in which the agent learns to select actions via continuous interaction with an environment, guided by feedback signals in the form of *rewards*. The agent’s objective is to maximize cumulative reward over time. The “deep” component refers to the use of deep neural networks to approximate policies or value functions allowing learning in complex, high dimensional environments. Traditional ML and classical control methods are often unsuitable for the “strategy under uncertainty” required in sailing. Supervised learning fails because there is rarely a comprehensive labeled dataset covering every combination of wave height, wind gusts, and sea state. Similarly, rule based controllers or classical PID schemes require explicit physical models, which struggle to capture the nonlinear interactions between the sails and dynamic ocean environment.

²The Vendée Globe is a non stop, round the world yacht race widely regarded as one of the most demanding sailing competitions.

³Closed-loop control continuously adjusts actions based on feedback from the current system state, e.g., via PID controllers.

⁴Course Made Good (CMG) is defined as the two dimensional projection of the boat’s velocity vector onto the target trajectory.

1.2 Principles of Sailboat Dynamics

Understanding the physical foundations of sailboat behavior is critical for designing effective RL agents. Sailboats are generally categorized into two types based on the number of buoyant hulls. **Monohulls** utilize a single hull with a heavy ballast to provide stability, while **Multihulls** achieve stability through a wider beam and multiple hulls. Regardless of body type, the interaction between the vessel and its environment is governed by several core structural components. Figure 2 illustrates these key architectural elements in real-world sailboats. **The Keel:** A fixed structure extending beneath the hull (Figure 2 A,B), the keel serves a dual function: generating a righting moment to resist heeling and producing hydrodynamic lift to minimize leeway (side-ways drift). **The Rudder:** The primary steering mechanism (Figure 2 D), the rudder creates a turning moment by deflecting water flow, allowing the sailor (or agent) to adjust the boat’s relative heading (β) and maintain the desired trajectory.



Figure 2: Sailing dynamics and components of a sailboat. (A & B) showing the keel (green) for lateral stability. (C) Vector diagram of CMG, course offset, and cross-track error (XTE). (D) Stern close-up showing the rudder (red) for steering.

The motion of a sailboat is governed by a dynamic equilibrium of Newtonian forces and fluid structure interactions. Propulsion is primarily generated through the **Coandă Effect**, in which airflow sticks to the curvature of the sail and is redirected backward. According to Newton’s Third Law, this deflection produces an equal and opposite reaction force, resulting in a total aerodynamic force. A component of this force provides the forward thrust (F_{thrust}) necessary to accelerate the vessel to a given velocity (V_{boat}). The net acceleration of the boat can be expressed:

$$F_{\text{net}} = F_{\text{thrust}} - F_{\text{drag}},$$

where F_{drag} includes both aerodynamic resistance and hydrodynamic friction. Maintaining directional efficiency requires a precise control: the keel converts lateral wind pressure into forward motion, while the rudder enables steering at the cost of minor “steering drag.” Mastery of this balance between air and water forces is the ultimate challenge that the RL agent must learn to optimize performance.

Beyond raw acceleration, the efficiency is measured by its progress toward a target destination. This is quantified by the Course Made Good (CMG), which represents the effective velocity projected onto the intended path. Given a course offset angle β —the angle between the boat’s heading and the target line CMG is defined as:

$$\text{CMG} = V_{\text{boat}} \cdot \cos(\beta) \quad (1)$$

Simultaneously, the agent must minimize the Cross-Track Error (XTE), the perpendicular distance from the current position to the target trajectory. If D denotes the distance traveled from the starting point, lateral deviation is expressed as:

$$\text{XTE} = D \cdot \sin(\beta) \quad (2)$$

By optimizing F_{net} to maintain speed while minimizing XTE, the sailors (agent) maximizes CMG along the most direct trajectory, achieving both efficient propulsion and precise navigation.

1.3 Problem Definition and Navigation Challenge

The primary objective of this project is to explore how DRL can extend traditional autopilot capabilities beyond simple set point maintenance, evolving them into intelligent systems capable of strategic navigation under uncertainty.

1.3.1 Simulation Environment: *BoatSimulator*

This study employs a high fidelity Python-based simulation environment called *BoatSimulator*⁵, built using the OpenAI Gymnasium framework to model boat dynamics in open sea conditions. For RL compatibility, the simulator is packaged within *boatsgym*, a wrapper compliant with the Gymnasium API. Key physical parameters governing the environment are summarized in Table ??, while the graphical user interface (GUI) and 3D visualization of the BoatSimulator are presented in Figure 3.

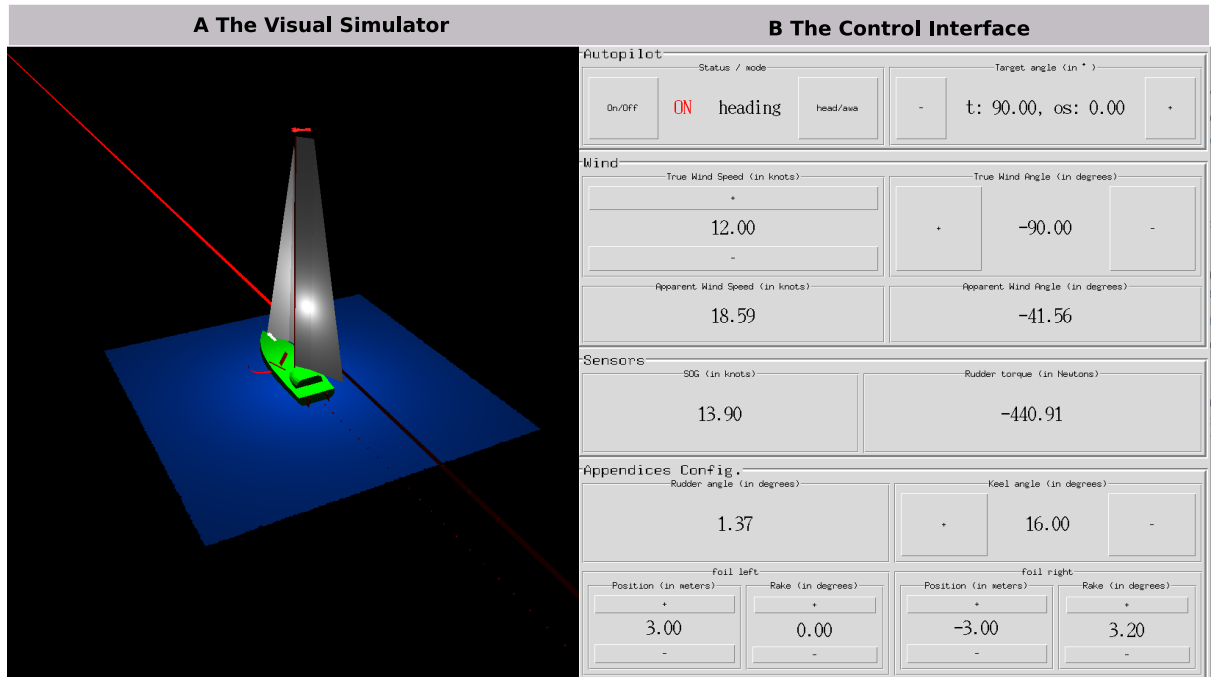


Figure 3: The *BoatSimulator* interface, featuring the system control dashboard (left) and 3D environmental viewport (right).

At the core of the navigation logic in the BoatSimulator is a classical Proportional-Integral-Derivative (PID) control loop which serves as the baseline for this study.

1.3.2 Baseline PID Performance and Navigation Limitations

Figure 4 illustrates a scenario in which the BoatSimulator’s PID controller attempts to maintain a target heading of 90° under a 12 kt wind. While the controller successfully stabilizes the boat’s orientation, a persistent lateral deviation (called leeway) remains. This Cross-Track

⁵Developed at Nautia

Error (XTE), visualized as the vertical red lines between the observed path and the target trajectory, arises because the PID controller only regulates angular heading and lacks awareness of the boat’s global position and effective forward progress. As a result although the heading is maintained, the sailboat drifts sideways relative to the intended trajectory, limiting its CMG.

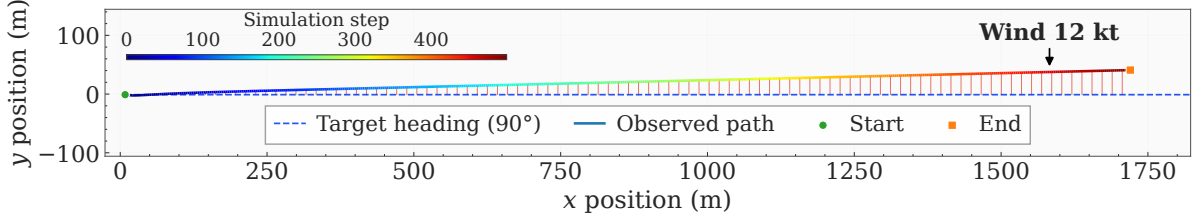


Figure 4: Trajectory toward 90° under 12 kt wind. The blue dashed line represents the target trajectory, while the vertical red lines indicate the accumulation of XTE despite stable heading control.

The performance implications of the PID controller’s limitations are further illustrated in Figure 5. The CMG reward representing the component of the boat’s velocity aligned with the target trajectory initially spikes as the vessel accelerates but ultimately stabilizes at a sub-optimal mean of 13.89 m/s. Corresponding control plots indicate that the heading error converges to a small non zero value, while the rudder angle stabilizes near 80° to counteract wind forces. Despite these stabilizations, the system fails to maximize forward velocity toward the goal.

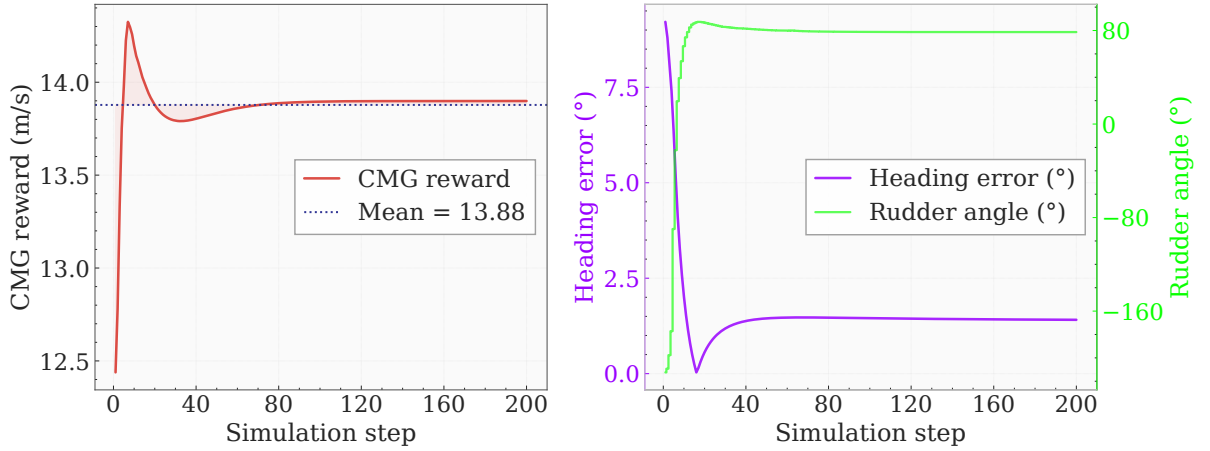


Figure 5: Baseline PID performance metrics. **Left:** CMG reward over time. **Right:** Heading error and rudder angle stabilization under constant environmental forcing.

Maintaining a constant heading alone is insufficient for maximizing forward progress (CMG). To address this limitation of classical closed-loop controllers, a DRL agent must learn to combine precise heading control with a strategy that proactively maximizes CMG, adapting continuously to stochastic environmental forces and moving beyond the constraints of simple set point regulation.

2 State of the Art in Autonomous Sailing Agents

2.1 Implementation Framework and Training Protocol

The state of the art RL agent was developed using the PyTorch deep learning library and the **Stable Baselines3** (SB3) framework. Each training experiment spanned 250,000 timesteps, and a multi-seed evaluation strategy ($N_{\text{seeds}} = 2$) was employed to ensure statistical robustness. Raw environmental observations, originally provided as hierarchical data structures, were processed through a **FlattenObservation** wrapper to produce a 1D feature vector. A **VecNormalize** wrapper was used to standardize observations and rewards to zero mean and unit variance. Additionally, a **CheckpointCallback** was invoked every 5,000 steps to archive model parameters and track agent performance throughout training.

To address the substantial computational and energy demands associated with long horizon RL training, we developed **NonDaemonicSubprocVecEnv** architecture. This framework enables synchronous parallel data collection across multiple environments, significantly reducing wall clock training time. Unlike standard vectorized environments, this custom implementation uses non-daemonic child processes⁶. This design not only accelerates policy convergence but also aligns with *Green carbon* objectives by maximizing hardware utilization and improving the overall energy efficiency of the computational pipeline.

Hyperparameter	Value
Learning Rate	$10^{-3} \rightarrow 10^{-5}$
Gamma (γ)	0.995
GAE Lambda (λ)	0.92
Batch Size	128
Entropy Coefficient	0.01
Number of Environments (N)	5

Table 1: Hyperparameter configuration used for training the PPO agent.

Table 1 summarizes the hyperparameter configuration used in this study. The learning rate was linearly decayed from 10^{-3} to 10^{-5} over the 250,000-step horizon to maintain training stability. A high discount factor ($\gamma = 0.995$) was chosen to prioritize long term mission objectives, such as maintaining optimal CMG, over short-term high-drag maneuvers. The GAE parameter ($\lambda = 0.92$) was tuned to reduce variance in the policy gradient, while an entropy coefficient of 0.01 was maintained to prevent premature convergence to sub optimal local minima. Training was parallelized across $N = 5$ environment instances to maximize throughput and computational efficiency. This setup minimizes idle time and ensures that each subprocess has dedicated access to the simulation’s physics and graphical contexts, facilitating stable and efficient learning.

2.2 Agent Architecture and Policy Network

The default decision making core of the RL agent is implemented as a Multi-Layer Perceptron (MLP) within an Actor-Critic framework. Both the policy network (π) and the value function (V) comprise two fully connected hidden layers of 64 neurons each with a non-linear activation functions. This configuration acts as a "reactive" baseline by mapping instantaneous state observations (such as wind angle, boat speed) directly to steering actions. While computationally efficient the standard MLP assumes that the current observation fully characterizes the environment. Foiling sailboats, however, operate in a regime where the temporal evolution of

⁶A non-daemonic child process operates independently of the main program’s exit status and is permitted to spawn its own sub-processes.

environmental variables is often as critical as their instantaneous values. The MLP treats each timestep independently, failing to capture the dynamics of wind gusts or shifting wave patterns. To remain on the foils and maximize CMG, the agent must distinguish between these transient turbulence and sustained changes in wind direction which necessitating an architecture capable of temporal reasoning.

2.3 Temporal Feature Extraction with 1D-CNNs

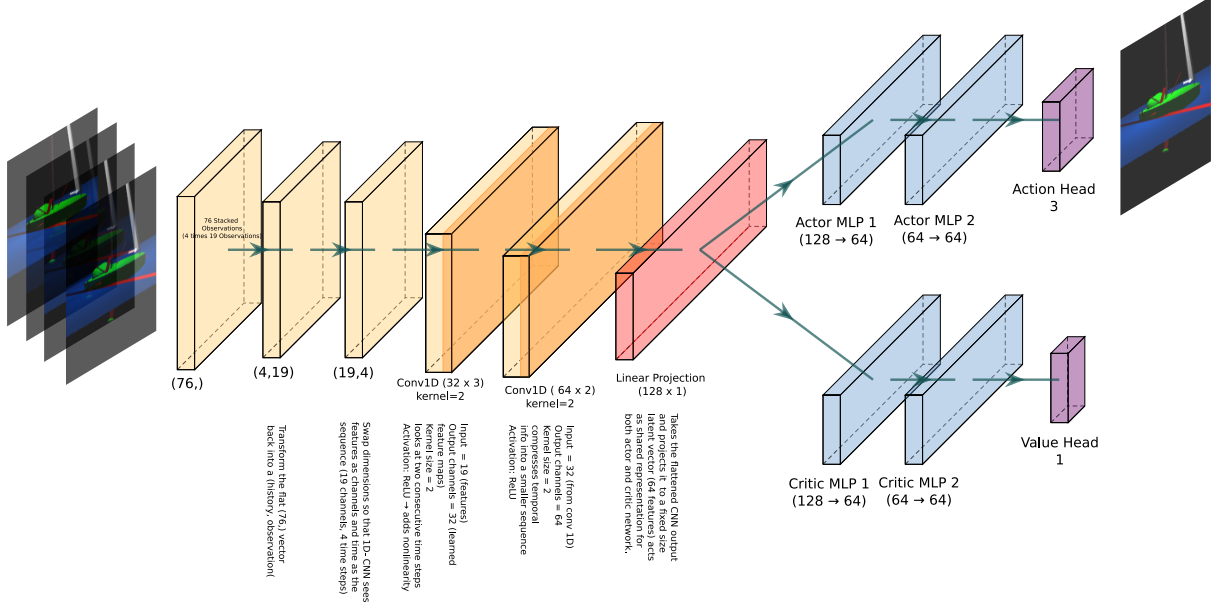


Figure 6: Proposed PPO Actor-Critic network featuring a history-based 1D-CNN feature extractor. The input comprises a temporal stack of 4 consecutive observations ($\sum_{i=0}^3 \mathbf{o}_{t-i}$). With each observation vector containing 19 physical features, the resulting 76-dimensional input is reshaped into a (19 channels $\times$ 4 time steps) tensor. The CNN architecture employs two Conv1D layers to extract temporal patterns, which are then projected into a 64-dimensional latent representation. This embedding is shared by the Actor and Critic MLP branches to determine the optimal 3D action vector and scalar state value, respectively.

To address the limitations of the reactive MLP, we introduce the **HistoryCNNExtractor**, a 1D Convolutional Neural Network (1DCNN) designed to capture temporal dependencies in sailboat (see Figure 6). By using a **VecFrameStack** of $n = 4$ frames, the agent receives a rolling history of the environment rather than isolated snapshots. The stacked observations from the last four timesteps, $\sum_{i=1}^4 \mathbf{o}_{t-i}$, are reshaped so that the 19 physical features are treated as channels and the four steps form the temporal dimension. This allows the convolutional kernels to slide along the time axis while observing all sensor features simultaneously at each temporal position.

Physically, the 1D-CNN functions as a *learnable differential operator*. The first convolutional layer, with a kernel size of 2, is mathematically analogous to a first-order difference operator in time:

$$\Delta x_t = x_t - x_{t-1}.$$

This allows the network to detect temporal trends such as the onset of wind gusts or sustained shifts in direction, providing the agent with richer information for anticipatory decision-making that improves CMG and overall sailing performance.

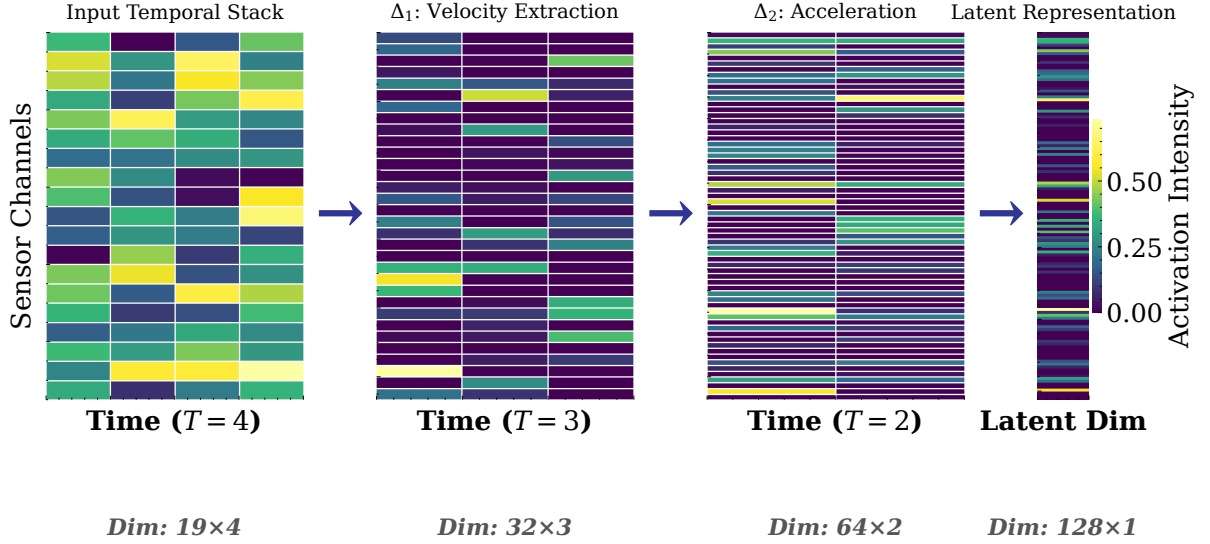


Figure 7: Hierarchical dataflow through the 1D-CNN feature extractor. **Panel 1:** Raw input matrix (19 features $\times$ 4 steps). **Panel 2:** After the first convolution ($k = 2$), the temporal dimension is reduced to 3; the 32 feature channels act as first-order differential operators. **Panel 3:** The second convolution reduces the temporal window to 2, processing velocity signals to extract second-order dynamics (acceleration). **Panel 4:** The latent representation, flattened to 64 channels $\times$ 2 steps = 128, provides a high-level physical abstraction for the Actor-Critic heads.

The network autonomously extracts *velocity-like* features, such as the rate of change in wind angle or boat speed. The second convolutional layer captures higher-order differences, effectively computing *acceleration-like* features. With a temporal window of four frames, the network achieves sufficient depth to stabilize these derivatives against simulation noise and environmental stochasticity. The resulting high-level features are flattened into a 128-dimensional latent embedding, which is passed to the Actor-Critic heads, providing a time-aware representation of the vessel’s momentum and trajectory. As illustrated in Figure 7, this architecture enables the agent to go beyond reactive control, making proactive navigation decisions informed by evolving environmental trends.

2.4 Training Protocol

Each training episode was limited to a maximum of 512 steps, with a control frequency of 2 Hz (the agent produced control decisions every 0.5 seconds). The action space is a discrete space allowing the agent to make incremental adjustments to the desired course (turn left, turn right, or maintain heading). Training was conducted under randomized environmental conditions to promote robustness and generalization. Wind speed was sampled uniformly between 10 and 18 knots, while target headings were drawn from upwind and reaching⁷ configurations ranging from 90° to 140° relative to the true wind direction. Navigation precision was enforced through a narrow target definition consisting of an 8° bearing width and a 1 m positional tolerance, encouraging accurate trajectory tracking.

2.4.1 Reward Design: CMG Optimization Objective

The primary learning objective was to maximize CMG. The reward function was defined as:

$$R = V_{\text{boat}} \cdot \cos(\theta_{\text{error}})$$

⁷wind comes from the side of the boat

where $V_{\text{boat}} \cdot \cos(\theta_{\text{error}})$ represents the component of velocity aligned with the target direction. The term C_{oob} corresponds to a significant penalty (50 units) applied when the sailboat exits its operational boundaries or loses foil stability, while C_{heading} enforces a maximum deviation constraint of 45° from the target heading.

This reward structure encourages sustained high-speed motion while maintaining directional accuracy and stable foiling behaviour.

3 Results and Discussion

This section presents the strategies used to obtain high performing agent and the evaluation methodology applied to compare the Baseline controller, the proactive MLP architecture and the reactive 1DCNN approach.

3.1 Training Convergence and Optimal Checkpoint Selection

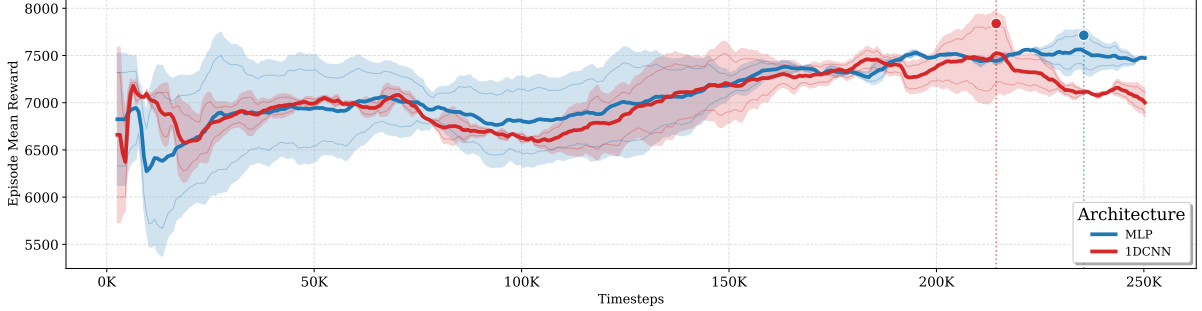


Figure 8: Training convergence and peak performance for MLP and 1DCNN architectures. Solid lines represent the exponential moving average (EMA, $\alpha = 0.8$) of episode mean reward across all seeds. Shaded regions denote standard deviation ($\pm 1\sigma$), while faint trajectories correspond to individual runs. Vertical markers indicate peak performance checkpoints.

As illustrated in Figure 8 both architectures shows a consistent upward trend in mean episodic reward, converging toward high performance plateaus exceeding 7,500. The 1DCNN demonstrates improved early stage stability and reaches its peak reward faster (approximately 215k timesteps) compared to the MLP baseline (approximately 235k timesteps). To ensure reliable model selection two stage checkpoint selection procedure was applied: (i) identification of the maximum smoothed reward ($R_{\max}$); (ii) temporal matching with the nearest saved checkpoint (stored at 5,000-step intervals). The selected checkpoints were subsequently evaluated across diverse environmental conditions.

3.2 Evaluation Framework and Performance Metrics

To evaluate generalization capabilities, both architectures were subjected to a deterministic evaluation suite across a combinatorial grid of 20 environmental configurations (Table 2). These scenarios are categorized into three distinct regimes:

- **Interpolation:** Headings of 95° , 115° , and 135° assess stability within the primary training distribution.
- **Extrapolation:** Headings at 85° and 145° (positioned 5° beyond the training boundaries) evaluate the graceful degradation of performance versus abrupt failure.
- **Stress Testing:** While wind speeds of 10, 14, and 18 knots represent trained conditions, the 20-knot scenario serves as a stress test to determine policy generalization under unobserved loads.

Feature normalization consistency was ensured by reloading the training-derived **VecNormalize** statistics. Performance was evaluated using multiple metrics: navigation precision (cross-track error and heading deviation), efficiency (CMG), and control stability (action slew rate).

Table 2: Evaluation Parameter Grid

Parameter	Testing Values
Target Headings (θ_{target})	85°, 95°, 115°, 135°, 145°
Wind Speeds (V_{wind})	10, 14, 18, 20 knots

3.3 Steady-State Performance Comparison

At the start of each navigation episode, the sailboat undergoes a transient phase before reaching stable dynamic behavior. To ensure a fair assessment of controller performance, these initial rising state were excluded from the analysis. Instead, steady state performance was evaluated using the tail segment of each episode where the control policy had converged to stable behavior. The steady state CMG was computed as

$$\text{CMG}_{\text{steady}} = \frac{1}{T} \sum_{i=201}^{512} \text{CMG}_i, \quad (3)$$

where the first 200 timesteps were discarded to remove transient effects and $T = N - 200$ denotes the number of samples used for steady state averaging. Figure9 shows the $\text{CMG}_{\text{steady}}$ comparison across test cases in Table 2.

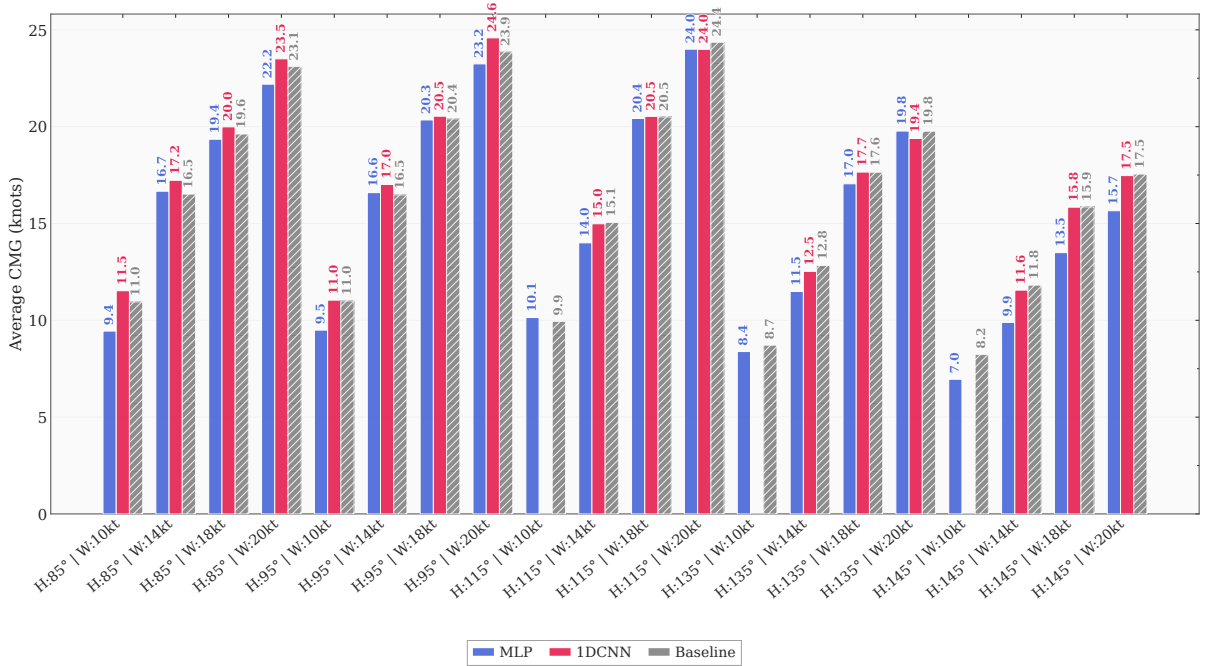


Figure 9: Steady-state CMG comparison across test cases. Performance differences are highlighted for out-of-distribution wind speeds (20 kts) and boundary headings (85°, 145°).

As shown in Figure 9, both RL architectures generally outperform the baseline PID controller, which consistently achieves the lower bounds of CMG across the majority of test cases (e.g., 11.0kts at $H:85^\circ | W:14\text{kt}$ compared to 17.2kts for the 1DCNN and 16.7kts for the MLP). Both RL agents demonstrate strong performance and clear generalization beyond the training envelope ($H \in [90^\circ, 140^\circ]$, $W \in [10, 18]\text{ kts}$). The 1DCNN emerges as the superior architecture for high-performance regimes, achieving the peak observed CMG of 24.6kts at $H:95^\circ | W:20\text{kt}$. Notably, the MLP exhibits significant performance degradation and policy instability under higher wind loads (18–20 kts), whereas the 1DCNN maintains stable operation at these higher limits.

However, a critical edge-case failure was identified for the 1DCNN in light-wind conditions (10kts) at headings $H : 115^\circ$, $H : 135^\circ$, and $H : 145^\circ$. In these scenarios, the 1DCNN policy frequently leads to “Out-of-Bounds” (OOB) conditions, resulting in prematurely terminated episodes and lower average CMG values (e.g., 7.7kts at $H : 135^\circ \mid W : 10$ kt) compared to the MLP (8.4kts) and baseline controller (8.7kts). This suggests that while the 1DCNN’s temporal feature extraction excels at maximizing velocity under high loads, it may over-correct in low-energy states where the physical limits of the foils provide less damping. The polar distribution in Figure 10 confirms that while the 1DCNN extends the performance frontier in the 120° to 150° heading range at high speeds, the MLP remains a more resilient, albeit less efficient, “fail-safe” option in marginal conditions.

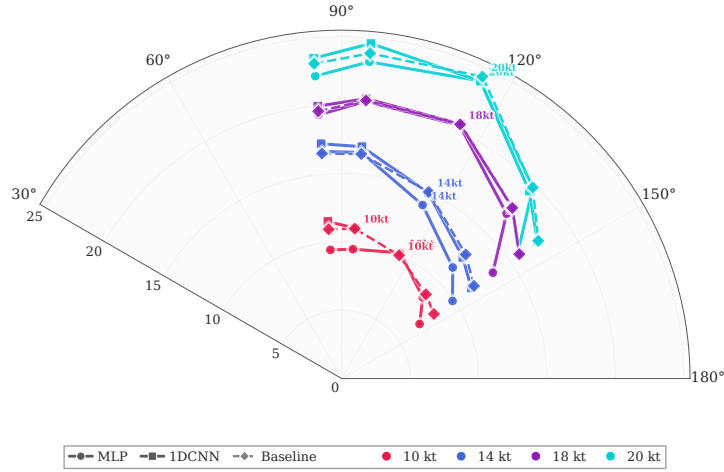


Figure 10: Polar performance distribution illustrating operational efficiency across headings. The 1DCNN maintains a broader and more consistent operational envelope.

3.4 Control Dynamics and Temporal Stability

To analyze the behavioral characteristics of the learned policies, we examine the action offset, defined as the numerical difference between the agent’s current action and its previous action ($\Delta a_t = a_t - a_{t-1}$). This metric serves as a proxy for control *jitter*: a high frequency of large offsets indicates aggressive or unstable steering behavior, whereas a concentration of small or zero offsets suggests smoother, more damped control resembling traditional PID-based steering. The command heatmaps in Figure 11 reveal distinct operational signatures between the architectures across the 512-step evaluation episodes. The Multi-Layer Perceptron (MLP) exhibits pronounced policy instability, characterized by frequent high-magnitude offsets oscillating between port and starboard extremes. This “chatter” suggests that the MLP responds primarily to instantaneous environmental fluctuations without effectively leveraging temporal context, particularly near operational boundaries where maintaining stability becomes challenging.

In contrast, the 1DCNN demonstrates a structured form of *adaptive jitter*. Under high-wind conditions (14–20 kts), steering corrections are concentrated around small-magnitude offsets, enabling the agent to maintain optimal foil lift and aerodynamic alignment while avoiding the

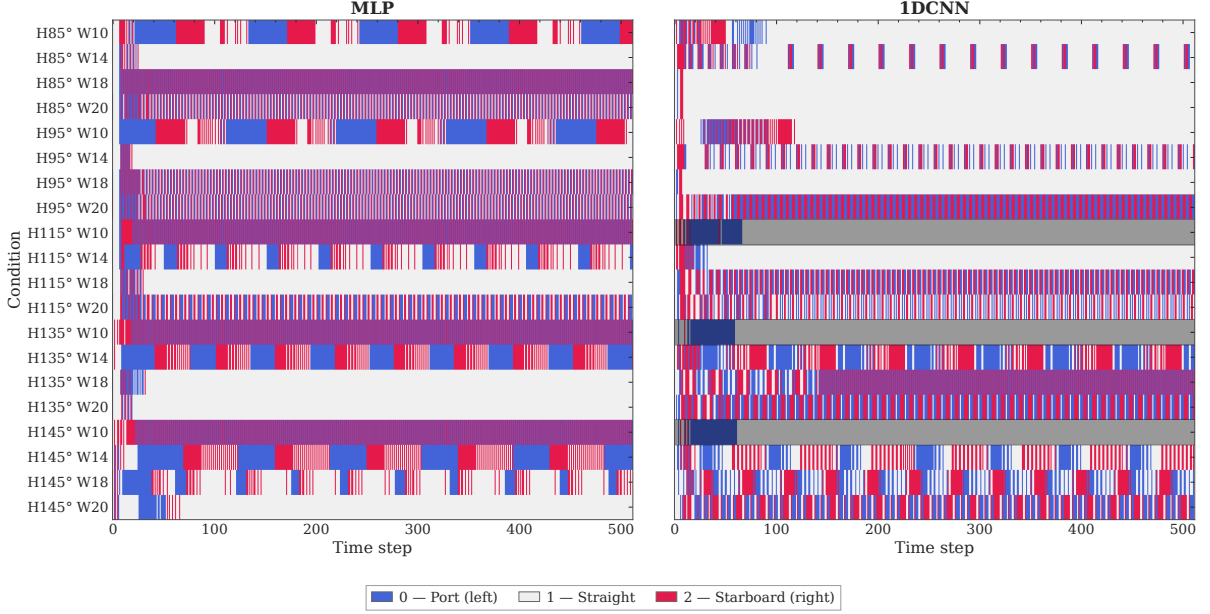


Figure 11: Spatiotemporal heatmap of steering action offsets for MLP (left) and 1DCNN (right) across varying heading (H) and wind (W) conditions. The color gradient represents the difference between consecutive actions ($\Delta a_t = a_t - a_{t-1}$), where 0 (white) indicates a maintained course and 2 (dark red/blue) indicates maximum rudder deflection.

large, energy-dissipating rudder oscillations observed in the MLP. However, this proactive control strategy introduces a trade-off in light-wind regimes (10 kts). At headings H:115°, H:135°, and H:145°, the 1DCNN’s micro-correction behavior can lead to “Out-of-Bounds” (OOB) failures, as insufficient wind pressure limits the physical damping of steering inputs. These results indicate that while temporal feature extraction in the 1DCNN significantly enhances robustness and performance under high-load conditions, its aggressive optimization may reduce stability in marginal environments where the vessel operates near physical limits. This highlights a key design consideration for autonomous foiling controllers: balancing proactive performance optimization with fail-safe reactive mechanisms during low-energy operating states.

3.5 Trajectory Analysis

The 2D trajectory plots (Figure 12) illustrate the spatial manifestations of the control policies discussed in the preceding sections. Under the boundary heading condition (H:85°), the Baseline PID controller maintains a highly linear path but lacks proactive lift management, resulting in lower velocity and sub-optimal drift with the longest time-to-target. The MLP exhibits low-frequency oscillations in light-wind scenarios (W:10 kt), producing a “wavy” trajectory that reflects the instability observed in the action-offset heatmaps, leading to increased drag and inefficient distance covered. In contrast, the 1DCNN demonstrates superior trajectory discipline, particularly under high-wind stress tests (W:20 kt), maintaining the most direct and stable line toward the target by using “adaptive jitter” to make micro-adjustments and prevent the large-scale deviations seen in the MLP. While the 1DCNN can experience abrupt termination in low-wind edge cases due to OOB failures, its performance under viable foiling conditions remains the most robust and energy-efficient among the controllers.

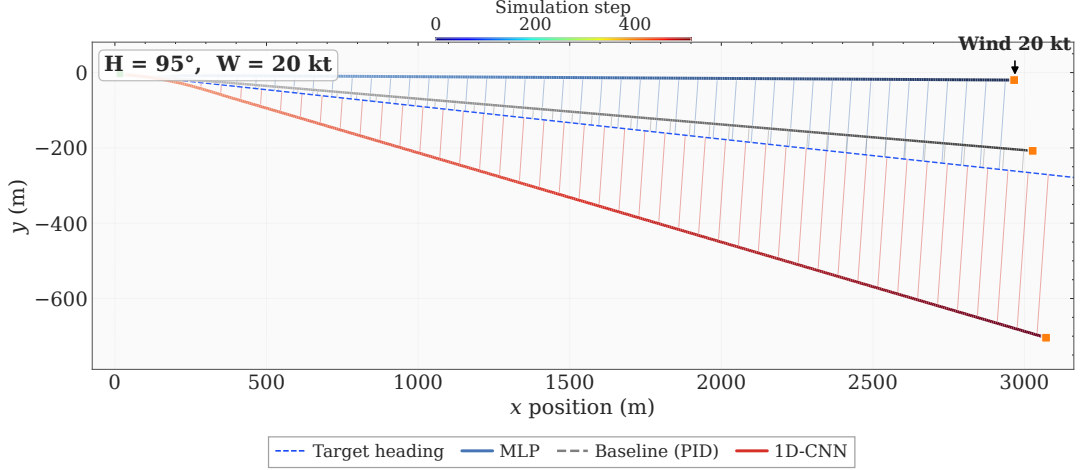


Figure 12: Distribution of action offsets (Δa_t) for MLP and 1DCNN across all test cases. The 1DCNN’s distribution is more concentrated around zero, indicating smoother control, while the MLP exhibits a wider spread, reflecting higher jitter.

4 Conclusion and Future Work

This study successfully developed and evaluated a 1D Convolutional Neural Network (1DCNN) architecture for the autonomous control of a vessel, comparing its performance against a standard Multi-Layer Perceptron (MLP) and a traditional Baseline controller. Our development of the 1DCNN focused on leveraging convolutional layers to enhance feature extraction from temporal environmental data, specifically targeting Course Made Good (CMG) optimization across diverse maritime conditions.

The results interpretation demonstrates that the 1DCNN is the most robust and high-performing model among those tested. It achieved a peak performance of 24.6 knots, notably under out-of-distribution high-wind conditions (20 kts), which underscores its superior ability to extrapolate learned physical dynamics beyond the training envelope. While the MLP showed competitive results in moderate, in-distribution scenarios, it suffered from significant operational instability and policy divergence in complex heading-wind configurations, such as the $H : 135^\circ$ case. Conversely, the 1DCNN maintained a stable and continuous performance profile, utilizing a learned "adaptive jitter" strategy to make high-frequency micro-corrections that maximized average speed. This indicates that the 1DCNN provides a more effective structural bias for maritime control tasks than traditional dense architectures.

Future work will focus on the following areas to further advance the system’s capabilities:

- **Dynamic Obstacle Avoidance:** Extending the 1DCNN policy to incorporate sensor fusion (e.g., LiDAR or Radar) for real-time navigation in environments with moving obstacles.
- **Multi-Objective Optimization:** Refining the reward function to balance maximum CMG with fuel efficiency and mechanical longevity by penalizing excessive rudder "jitter."
- **Transfer Learning for Variable Vessel Dynamics:** Investigating the portability of the learned 1DCNN weights to different hull designs and propulsion systems to determine the universality of the learned control laws.
- **Real-World Deployment:** Transitioning from high-fidelity simulations to hardware-in-the-loop (HIL) testing and eventually full-scale maritime trials to validate the 1DCNN’s performance under stochastic real-world sea states.

A Complete Experimental Results Table

Hdg (°)	Wind (kt)	MLP			PID			1DCNN (This work)		
		CMG	Err (°)	XTE (m)	CMG	Err (°)	XTE (m)	CMG	Err (°)	XTE (m)
85	10	9.44	10.40	155.12	10.95	1.25	21.31	11.53	6.10	108.46
85	14	16.67	0.60	15.41	16.51	1.54	39.66	17.23	3.75	100.85
85	18	19.35	2.39	72.49	19.62	0.94	28.84	19.99	0.99	30.94
85	20	22.19	4.65	162.10	23.11	1.05	38.08	23.50	0.88	32.36
95	10	9.49	11.90	179.37	11.00	1.32	22.67	11.04	15.58	283.57
95	14	16.59	0.62	15.95	16.48	1.55	39.78	17.02	3.95	105.38
95	18	20.35	1.70	54.21	20.40	0.97	30.99	20.54	0.02	0.58
95	20	23.24	4.61	168.55	23.87	1.10	40.94	24.59	7.71	299.62
115	10	10.14	6.57	104.95	9.94	1.47	22.78	N/A	N/A	N/A
115	14	14.00	11.88	266.68	15.05	1.54	36.16	14.98	2.01	48.00
115	18	20.42	2.07	66.18	20.50	1.08	34.57	20.53	7.10	230.81
115	20	24.00	5.56	209.87	24.36	1.18	44.99	23.99	7.92	302.95
135	10	8.39	10.32	137.14	8.71	1.64	22.26	N/A	N/A	N/A
135	14	11.49	19.33	363.91	12.83	1.61	32.11	12.53	0.41	7.74
135	18	17.05	11.39	309.64	17.64	1.06	29.34	17.65	0.15	3.38
135	20	19.77	2.07	63.96	19.76	1.08	33.37	19.39	4.49	137.72
145	10	6.95	11.90	132.60	8.23	1.76	22.41	N/A	N/A	N/A
145	14	9.89	22.01	360.30	11.81	1.69	30.97	11.56	7.62	138.39
145	18	13.49	23.97	546.05	15.85	1.13	27.78	15.84	2.86	70.94
145	20	15.66	18.04	463.74	17.55	1.08	29.46	17.47	0.20	5.49

Table 3: Systematic comparison of navigational metrics across architectural groups, sorted by heading and wind intensity.

Hdg (°)	Wind (kt)	CMG Values			% Improvement vs PID	
		MLP	PID	1DCNN	MLP	1DCNN
85	10	9.44	10.95	11.53	−13.77	+5.27
85	14	16.67	16.51	17.23	+0.92	+4.33
85	18	19.35	19.62	19.99	−1.36	+1.90
85	20	22.19	23.11	23.50	−3.95	+1.72
95	10	9.49	11.00	11.04	−13.71	+0.36
95	14	16.59	16.48	17.02	+0.71	+3.28
95	18	20.35	20.40	20.54	−0.28	+0.67
95	20	23.24	23.87	24.59	−2.61	+3.02
115	10	10.14	9.94	N/A	+2.03	N/A
115	14	14.00	15.05	14.98	−6.98	−0.44
115	18	20.42	20.50	20.53	−0.39	+0.14
115	20	24.00	24.36	23.99	−1.45	−1.48
135	10	8.39	8.71	N/A	−3.73	N/A
135	14	11.49	12.83	12.53	−10.47	−2.35
135	18	17.05	17.64	17.65	−3.36	+0.06
135	20	19.77	19.76	19.39	+0.07	−1.87
145	10	6.95	8.23	N/A	−15.55	N/A
145	14	9.89	11.81	11.56	−16.25	−2.15
145	18	13.49	15.85	15.84	−14.89	−0.07
145	20	15.66	17.55	17.47	−10.78	−0.44

Table 4: Percentage CMG improvement of MLP and 1DCNN over the PID baseline. N/A indicates the model failed for that condition. Positive values (red) indicate the model outperforms PID.

B CMG Improvement Over Baseline

C Complete Trajectory Comparison Across Architectures and Wind Conditions

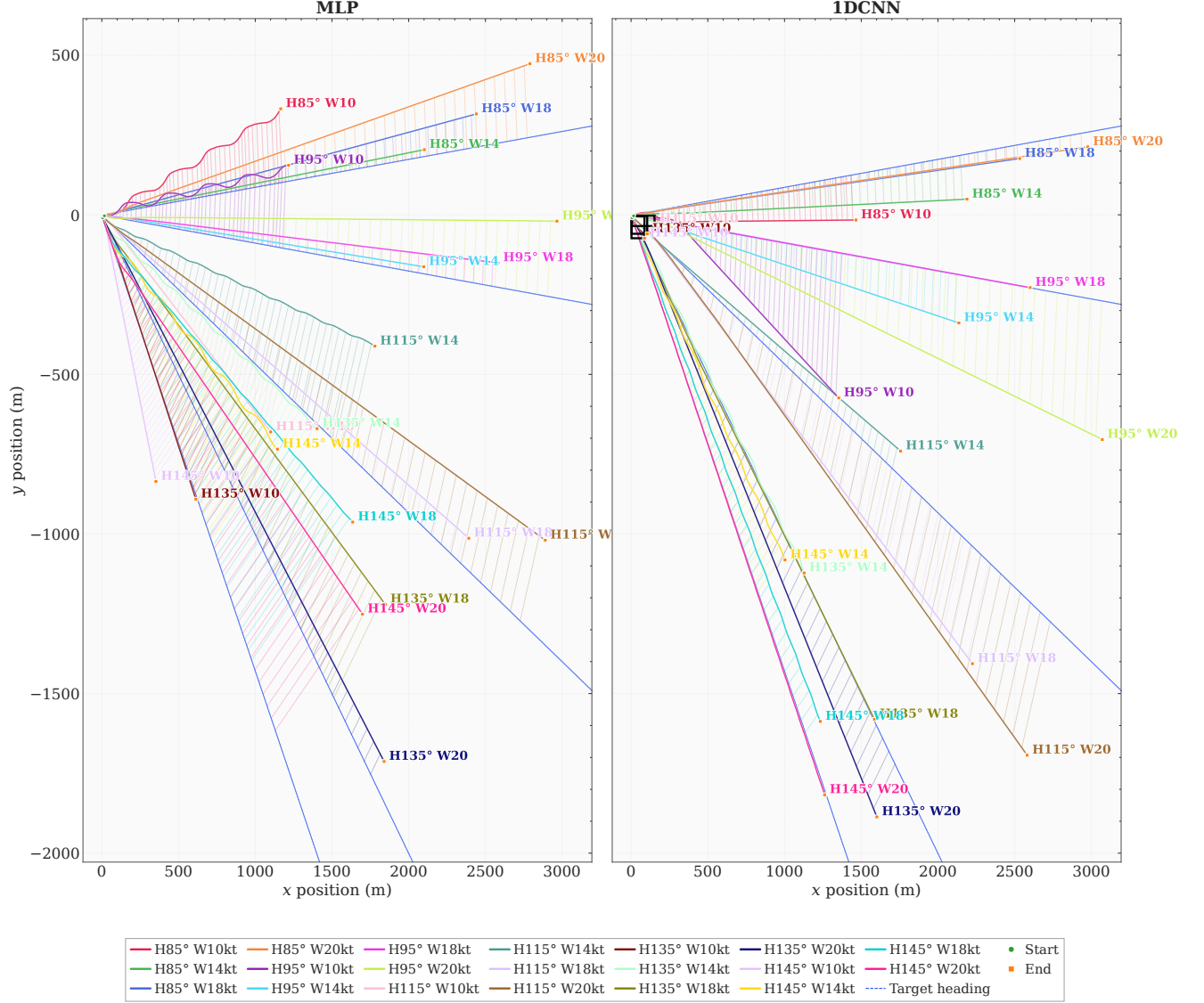


Figure 13: Trajectory comparison across different architectures and wind conditions.

The trajectory plots provide a qualitative comparison of the steering behaviour exhibited by the three control architectures under varying wind conditions. The Baseline controller produces the most linear and geometrically smooth trajectories; however, it demonstrates increased drift under high-wind scenarios, suggesting limited adaptability to rapidly changing environmental forces. The MLP-based controller exhibits noticeable instability, particularly in low-wind cases (e.g., H:85° | W:10 kt), where large oscillations and inefficient path deviations indicate insufficient control damping and reduced trajectory efficiency.

In contrast, the proposed 1DCNN architecture generates the most consistent and robust trajectories across the entire evaluation grid. Even under challenging conditions such as W:20 kt, the 1DCNN maintains tighter path adherence with reduced lateral deviations compared to the MLP model. These results suggest an improved capability to capture and respond to the nonlin-

Autonomous sailing is not the automation of navigation — it is the learning of strategy under uncertainty.

ear aerodynamic and hydrodynamic dynamics inherent to high-performance foiling sailboats.